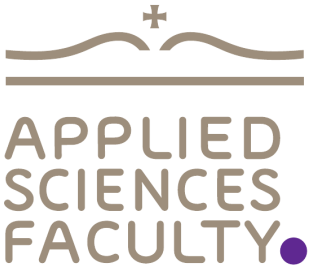
C++ — додаток. C++ в Embedded





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

C++ на мікроконтролерах

- Із C++ в Embedded, особливо на мікроконтролерах[1] (MCU) пов'язано багато міфів та забобон.
- Хоча деякі застереження обґрунтовані, більшість – саме міфи та забобони. Шкоди із них більше ніж користі[2].
- Embedded системи надзвичайно різні. MIPS із Linux на борту та 1 Гб пам'яті у рутері теж називають embedded-пристроєм. Надалі, для визначеності, говоритимемо про менш-більш звичні MCU.

[1] Для яких характерні малі розміри RAM та Read-only пам'ять програм теж не особливо великого розміру.

[2] Так, я знаю, що Лінус Торвальд думає по іншому.

Особливості Embedded-середовища

- Малі об'єми RAM: від лічених кілобайт до перших сотень кілобайт[3].
- Пам'ять програм – read-only[4]. Її об'єм – від кілобайт до перших мегабайт.
- Зазвичай, відсутні перезаписувані носії типу HDD/SSD[5].
- Відносно слабкий процесор[6] – від перших МГц до перших сотень МГц, без суперскалярності та OoO.
- Потреба Real time – поширена. Часто навіть hard real time.
- Часто – операційна система відсутня. Коли присутня – зазвичай, вона надає лише переключення задач та примітиви обміну інформації між ними та синхронізації.
- Часто – безперебійна їх працездатність критична.
- Часто використання динамічної пам'яті неможливе з тих чи інших причин.
- Величезне розмаїття архітектур і компіляторів.

[3] Крихітні MCU мають менше, ATTiny13 – 64 байти RAM, існують також MCU типу ATTiny10 чи HD44780, які взагалі не мають звичного RAM.

[4] Хоча сучасні MCU часто підтримують можливість само-програмування.

[5] Хоча SD-картку підключити часто можна.

[6] Хоча, FPU та DSP розширення стають все популярнішими.

Навіщо C++?

- Відповідь проста – зручність[7].
- Чим більш високорівнева абстракція, тим простіше з нею працювати. Питання лише її ціни[8].
  - ◇ C++ надає дуже потужні механізми побудови абстракції.
- Найкраща помилка – та, яка сталася під час компіляції. Трохи гірша – коли код зразу аварійно завершується, принаймні – за виконання певної операції з ним. Причина – чим чіткіше маніфестується помилка, тим легше її виявити та виправити.
  - ◇ C++ дозволяє контролювати адекватність коду ще на етапі компіляції. Наприклад, неприйнятні значення пінів, констант чи бітмасок може виявляти ще компілятор[9].
    - ◇◇ При чому, це може бути чистий і стандартний C++ без зовнішніх кодогенераторів абощо.

---

[7] Порівняння – відносно C та асемблера. Інші мови на MCU не приживаються. Єдина надія на розширення списку – rust.

[8] І протікання абстракції.

[9] Це потребує зусиль, звичайно. Але це можливо зробити у важливих випадках. Яких в Embedded багато. Особливо, враховуючи, що автоматизація тестування ускладнена.

Які мінуси?

- Людський фатор: C++ допускає написання дуже поганого коду. Скажімо так, на ньому це зробити легше, ніж на C.
  - ◇ З іншого боку, на асемблері – ще легше, ніж на C.
- Фактор складності: C++ багато що робить під капотом. Без глибокого розуміння це стає проблемою на системах із жорстким обмеженням на ресурси та серйозними вимогами до надійності.
  - ◇ Це, звичайно, теж варіант людського фактору – як і все в програмуванні, яким займаються люди. Але C в тому плані значно кооперативніша мова – під капотом майже нічого[10].
- Компілятори для цього розмаїття платформ дуже відрізняються за якістю. Зокрема, підтримка сучасних засобів C++, поміж яких багато спрощують роботу із MCU, часто, обмежена.
  - ◇ З іншого боку, GCC є практично всюди. Не завжди найновіший, щоправда...
- Підсумовуючи: бар'єр входження вищий, ніж для C, а способів вистрелити собі в ногу – на порядок-другий більше.
- Ремарка: діалект під назвою Embedded C++ – відверто невдалий. Цю думку розділяє більшість C++ спільноти. Поміж справді сумнівних в Embedded засобів[11], викинуто й дуже корисні – аж по простори імен та шаблони.
  - ◇ З іншого боку, саме цим діалектом можна писати драйвери для MacOS, див. I/O Kit.

[10] Це не зовсім правда, але в порівнянні з C++ – хороше наближення.

[11] Таких, як виключення – які й так більшість компіляторів вміють "викидати" із мови.

Що із цим робити?

- Власне, те ж, що й завжди – якісно програмувати.

1. Розуміти своє середовище і задачу – які вимоги, які обмеження.
2. Розуміти, як функціонують засоби C++.
3. Пам'ятати – чим простіше, тим краще.
4. Краще досконало володіти C++, але якщо поки ні – починайте інкрементально, із вашого звичного підходу C та розширюйте його[12].

- Не завжди очевидний плюс від адекватних абстракцій – розробка спрощується якісно, дозволяючи реалізовувати речі, мало доступні "з нуля".
- І все таке.

[12] В загальному випадку ця порада шкідлива. Мова лише про впровадження C++ у проекти для MCU.

Ціна різних засобів C++

- Поділимо всі засоби на декілька класів.
- **Безкоштовні** – за правильного використання не спричиняють накладних витрат самі по собі.
  - Звичайно, вони можуть бути використані помилково і тоді перестають бути безкоштовними. Але так завжди буває...
- Інструменти, які спричиняють накладні витрати, але якщо їх функціонал потрібен, вони можуть бути виправданими.
  - Канонічний приклад – віртуальні функції.
- Засоби, саме існування яких не є безкоштовним – якщо вони підтримуються, паде швидкодія чи збільшується об'єм коду, навіть якщо ви їх не використовуєте.
  - В основному, це виключення та RTTI.

Безкоштовні засоби мови C++

- Все із C – за означенням.
- Посилання (reference). За акуратного використання їх ціна може бути від'ємною.
- Класи без віртуальних функцій та їх методи. Зокрема – конструктори і деструктори
- Шаблони.
- Перевантаження функцій та аргументи за замовчуванням.
- Простори імен.
- Успадкування. Включаючи множинне, але з нюансами.
- Керування динамічною пам'яттю за допомогою new та delete[13]
- Окремі інструменти, такі як constexpr, можуть мати **від'ємну ціну**.
- ⋆ Кожним із цих засобів, звичайно, можна скористатися помилково.

[13] Включаючи placement new, operator new – як аналог malloc() і new(nothrow) як стандартне вираження відмови від виключень.

Формально – не безкоштовні засоби мови

- Віртуальні функції – vptr, vtable та опосередковані виклики.
  - Однак, якщо вам потрібен їх функціонал – навряд, чи у вас вийде краще! А більшість MCU не суперскалярні абощо і не особливо швидші за свій RAM – опосередкований виклик дешевий.
- RTTI – внаслідок зростанню vtable.
  - З іншого боку, суб'єктивна думка доповідача – на MCU використання RTTI, як мінімум, насторожує.
- Множинне успадкування з віртуальними базовими класами та інші подібні жахіття – не будемо заглиблюватися.

Достатньо дорогі засоби мови

- В цій категорії – хіба виключення.
- Виключення C++ – однозначні парії в світі C++ для Embedded.
- Вони потребують багато коду підтримки – 50 і більше кілобайт за сам факт на ARM Cortex Mx[14]
- Їх важко використовувати у системах, що вимагають передбачуваності поведінки, зокрема – real-time системах.
- В цілому – уникайте.
- Важлива **ремарка**: потреба обробляти помилки нікуди не зникає навіть без виключень.
  - ◇ Скотт Меєрс у своїй "Effective C++ in an Embedded Environment" згадує, що обробка помилок з використанням кодів завершення збільшує код на 15-40%.

[14] http://indrekis2.blogspot.com/2015/11/c-arm-gcc-stm32-coide.html.

Безкоштовні та недорогі засоби стандартної бібліотеки – 1

- Засоби стандартної бібліотеки C не стали дорожчими. Хоча, безкоштовні не всі.
- Стандартні алгоритми із STL практично завжди ефективні і не призводять до code bloat. Наприклад, std::sort() швидший за qsort() із C. Але потребують уваги.
- std::tuple, std::array – самі по собі безкоштовні.
- std::list, std::forward_list – теж, якщо пам'ятати, що це, все ж, зв'язні списки і що виділення вузлів відбувається динамічно.
- std::vector – складніший. Він, за замовчуванням, у випадку вичерпання пам'яті, алокує вдвічі більше. На MCU може бути проблемою, через відсутність стандартних засобів впливати на "коефіцієнт алокації", доводиться акуратно застосовувати метод reserve().
- Адаптери stack, queue, priority_queue – безкоштовні самі по собі.
- std::string достатньо ефективний, зокрема завдяки small string optimizations і можливому reference counting[15]. Але резервують додаткову пам'ять. Коефіцієнт залишається на розсуд розробників стандартної бібліотеки – на відміну від вектора.

[15] Нестандартному починаючи із C++11.

Безкоштовні та недорогі засоби стандартної бібліотеки – 2

- Розумні вказівники – недорогі, якщо пам'ятати про динамічне виділення блоку для reference counting.
- Асоціативні контейнери – середньої вартості. Потребують помітної runtime-підтримки, але, часто, кращі ніж "самопальні", якщо вам потрібне такий функціонал.
- Floating point можу бути дорогим, якщо не немає апаратної підтримки (FPU).

Дорогі засоби стандартної бібліотеки

- Бібліотека вводу-виводу iostream – неймовірно громіздка і рідко адекватна для MCU.
  - Вона й на "великих" системах дискусійна.
- Підтримка локалізації в стандарті, окрім загальної дискусійності та громіздкості для MCU теж підтримується на них так собі.
  - Потребу в локалізації Embedded-пристроїв, зазвичай, практичніше вирішувати іншими засобами.

Інше

- Атомарні змінні та засоби багатопоточності, зазвичай, підтримуються погано.
  - Їх роль можуть брати на себе задачі та засоби комунікації RTOS.
  - Але, якщо дуже потрібно – всілякі std::thread і std::mutex можна реалізувати поверх засобів RTOS.

На що звертати увагу

- Віртуальні функції: пам'ятайте про опосередкованість їх виклику. Зважуйте необхідність.
- New – виділення пам'яті плюс виклик конструктора.
  - Якщо потрібно створити об'єкт за вказаною адресою – часта задача на MCU із їх memory mapped I/O, скористайтеся placement new.
  - З іншого боку, якщо потрібна динамічна пам'ять – допоможе оператор new (на противагу malloc()).
  - Хоча GCC та інші компілятори для MCU вміють викидати підтримку виключень (ключ -fno-exceptions), пам'ятайте, стандартний new без виключень – new(nothrow).
- Аналогічно, delete викликає деструктор і звільняє пам'ять. Оператор delete – лише звільняє.
- Конструктор – за адекватного вибору абстракції та акуратної її реалізації, це мінімальний достатній набір ініціалізації об'єкту. Його виклик – необхідність і добре, коли компілятор може автоматизувати цей процес.
  - Тому краще акуратно проектувати абстракції програмного забезпечення, користуючись цією автоматикою, замість того, щоб все робити вручну.
- Те ж стосується деструкторів.

- Однак, пам'ятайте, що конструктор та деструктор викличе (в FIFO порядку) всі конструктори/деструктори базових класів та його членів.
- Аргументи за замовчуванням – пам'ятайте, що вони передаються неявно за кожного виклику функції. Пильнуйте, щоб це не було дорого.
- Тимчасові об'єкти! Потенційне джерело серйозних проблем. Якщо a, b і c об'єкти якогось класу із перевантаженим operator+, то c = a + b, ймовірно[16] створить мінімум один тимчасовий об'єкт.
- Шаблони можуть призводити до code bloat – роздування коду.
  - ◇ Вирішення – приватна реалізація із успадкуваннями, службовими функціями, низькорівневими void* тощо, обгорнута в зручні шаблони. Це – краща альтернатива до "ну ті шаблони, вони код роздувають, зроблю все на сирих вказівниках".
- Inlinging – потужний інструмент, але теж може призводити до збільшення розмірів коду.
- Компілятор може переставляти private, protected та public частини класу – і його layout в пам'яті може стати для вас неочікуваним. POD – важлива концепція.

[16] Запобігти цьому можна хитрим метапрограмуванням із використанням template expressions. Див. Eigen та історичну blitz++.

- Вимоги до прототипу обробників переривань жорсткі – пам'ятайте про name mangling і про те, що не статичним методам неявно передається посилання на сам об'єкт.
- ROM-ablity безперечно підтримується C++, але теж може потребувати уваги. Наприклад, на AVR часто трапляється копіювання зайвого із ROM/EEPROM в RAM – всіляких там константних стрічок.

Приклади

- Щоб не бути багатослівним – вся екосистема Arduino базується навколо бібліотек C++.
  - ◇ Дуже скромний препроцесор над C++ у Arduino навряд чи варто називати "мовою wiring".
  - ◇ Якість частини тих бібліотек та компроміси вибрані при створенні стандартних бібліотек Arduino – дискусійні. Але їх однозначним плюсом є доступність та простота у використанні. Все ж, бар'єр входження в Embedded без них – неймовірно крутий[17].
- Зупинюся на іншому – приклад використання шаблонів, як альтернатива до обраного Arduino шляху: `http://indrekis2.blogspot.com/2016/02/hc-sr04-gpioc.html`.

[17] Для снобів: трапилося колись в коментарях на Хабрі: "Навіщо то Ардуїно [як демоплата] – береш текстоліт, розводиш простеньку плату, фоторезист/LUT, все таке, витравлюєш, запаюєш і можеш працювати." Серйозно? Це шлях для людини, яка хоче спробувати MCU програмувати?

Середовище, у якому виконується код

- На відміну від програм для "великих" ОС, код на MCU не може розраховувати на наявність ОС, або, принаймні, на те, що вона надає багато всіляких засобів та API.
- Для впорядкування, в C++11 ввели поняття freestanding та hosted систем.
- **Надалі говоритимемо про конкретний приклад – GCC для ARM Cortex M.** Подробиці для інших мікроконтролерів чи компіляторів відрізнятимуться, хоча загальна схема – та ж.
- Runtime-код і C і C++ потребує, рано чи пізно, низькорівневих засобів для роботи із апаратурою.
  - ◇ Для C – див. Newlib та реалізацію функцій-системних викликів.
  - ◇ Засоби підтримки C++ в GCC – бібліотека libsupc++[18]. Вона теж покладається на низькорівневі функції. В ідеалі, проекти C++ коду для MCU, створені із шаблонів чи згенеровані іншими засобами (типу генератора проектів STM32CubeMX) мали б містити все необхідне. На практиці – це далеко не завжди так.
- **Ремарка**: протягом останнього десятиліття стан підтримки C++ в середовищах розробки якісно покращився – зазвичай все працюватиме "з коробки". Але бути уважним доводиться!

[18] Варіант від clang – libc++abi, див. https://libcxxabi.llvm.org/spec.html.

Початок

- Коли ARM Cortex M починає працювати, він бере значення із адреси 0x0 флеш-пам'яті і кладе його в SP – регістр стеку.
- Після цього – передає керування за адресою, яка міститься в комірках флешу, починаючи із 0x4 – до обробника переривання із номером 1, Reset.
- Цей обробник повинен підготувати середовище для успішного функціонування main().
- Подивимося на типовий такий обробник:

```
  .weak  Reset_Handler
  .type  Reset_Handler, %function
Reset_Handler:
  ldr   sp, =_estack       /* set stack pointer */
```

```
/* Copy the data segment initializers from flash to SRAM */
  movs  r1, #0
  b  LoopCopyDataInit

CopyDataInit:
  ldr  r3, =_sidata
  ldr  r3, [r3, r1]
  str  r3, [r0, r1]
  adds  r1, r1, #4

LoopCopyDataInit:
  ldr  r0, =_sdata
  ldr  r3, =_edata
  adds  r2, r0, r1
  cmp  r2, r3
  bcc  CopyDataInit
  ldr  r2, =_sbss
  b  LoopFillZerobss
/* Zero fill the bss segment. */
FillZerobss:
  movs  r3, #0
  str  r3, [r2], #4

LoopFillZerobss:
  ldr  r3, = _ebss
  cmp  r2, r3
  bcc  FillZerobss
```

- Далі ще цікавіше:

```
/* Call the clock system intitialization function.*/
  bl  SystemInit
/* Call static constructors */
    bl __libc_init_array
/* Call the application's entry point.*/
  bl  main
  bx  lr
```

- Що це за `__libc_init_array`? Вона викликає конструктори для глобальних та статичних об'єктів. Це не обов'язково конструктори в сенсі C++, це спеціальні функції, створені GCC для ініціалізації. Наприклад, без них:

```
int my_a = 7;
int my_b = my_a;
```

де обидві змінні глобальні, може виникнути ситуація, коли my_b == 0! Тому що встановлювалася вона в конструкторі.

- GCC створює все необхідне, щоб їх викликати, але якщо startup-код цього не робить – поведінка буде більш ніж неочікуваною.
- Те ж стосується і деструкторів (у широкому сенсі). Але на MCU знищувати глобальні об'єкти доводиться рідше – виходити з програми немає куди.
- Подробиці див. http://indrekis2.blogspot.com/2015/11/

Керування виділенням пам'яті

- New та delete часто надаються для контролерів в урізонаму або взагалі дивному стані.
- Іноді доводиться реалізовувати самостійно:

```
void* operator new (std::size_t size, const std::nothrow_t& n){
 return malloc(size);
}

void* operator new[] (std::size_t size, const std::nothrow_t& n){
 return ::operator new(size, n);
}

void* operator new (std::size_t size){
 void *ptr = ::operator new(size, std::nothrow);
 if (!ptr) {
#ifdef USE_RTTI_AND_EXCEPTIONS
   throw std::bad_alloc();
#else
   std::terminate();
#endif
 }
 return ptr;
```

libsupc++ та libstdc++

- Runtime-код стандартної бібліотеки знаходиться в libstdc++.
- Якщо ви використовуєте, скажімо, std::vector<> без неї, вам доведеться (для GCC) надати реалізацію деяких функцій, інакше лінкер скаржитиметься на їх відсутність:

```
namespace std{
 const nothrow_t nothrow;
 void __throw_length_error(char const* str){
  puts( str );
  std :: terminate ();
 }
 void __throw_bad_alloc() {
  std :: terminate ();
 }
}
void std :: terminate(){
 abort ();
}
```

- При тому, функції типу `__throw_length_error` – weak-символи, їх замінить ваша реалізація, навіть якщо ви підключили libsupc++ та libstdc++, а terminate() повинна бути лише одна.

Що робити, коли викликали відсутні функції

- Без libsupc++ вам доведеться надати функції реакції на виклик чисто віртуальної функції та видаленої функції:

```
extern "C" void __cxa_pure_virtual(void) {
    // Можна повідомити про помилку, через USART,
    //  чи засобами semihosting.
    // Наприклад:
    // puts("Pure virtual function called");
    std::terminate();
}

extern "C" void __cxa_deleted_virtual(void) {
    std::terminate();
}
```

- Так, добитися їх виклику нетривіально, але "захист від дурня проти винахідливого дурня безсилий" і лінкер може їх потребувати. Іноді може ні – як компілято вирішить собі.

Локальні статичні змінні

- Створення локальних статичних змінних повинне бути захищеним від data race.
- GCC використовує для цього функції `__cxa_guard_acquire()`, `__cxa_guard_release ()`, `__cxa_guard_abort ()`.
- Попросити його не захищати їх можна ключем `-fno-threadsafe-statics`.
- Або створити. Безглузда реалізація виглядатиме так:

```
#include <cxxabi.h>
using __cxxabiv1::__guard;

int __cxa_guard_acquire(__guard *g)
{
    return !*(char *)(g);
};
void __cxa_guard_release (__guard *g) {
    *(char *)g = 1;
};
void __cxa_guard_abort (__guard *)
{};
```

- Повноцінна повинна покладатися на бар'єри пам'яті та примітиви RTOS вашої платформи.

Асоціативні контейнери та все решта

- Якщо використання простих контейнерів, типу std::vector чи std::list вимагатиме, без участі libstdc++, додати лише жменю простих функцій – на відсутність яких скаржитиметься лінкер, то вручну створити все необхідне для std::map – якісно важче.
- Якщо ви підключите libstdc++, все буде автоматом.
- Однак, при тому може різко зрости використання пам'яті, як RAM, так і ROM. Наприклад, алокація стандартними контейнерами, включаючи std::vector може почати відбуватися цілими сторінками. Код теж сильно зросте.
- Вся надія на лінкер, який зможе викинути зайве.
- Зважуйте – наскільки результат вартуватиме зусиль.
- Зважуйте ризики зниження передбачуваності такого коду.

Freestanding та hosted реалізації – C++11

- C++11 розділив два можливих реалізації:
  - ◇ freestanding – "незалежна", на вбудованих системах, без "масованої" runtime-підтримки,
  - ◇ hosted – поверх "великої" ОС, з повною підтримкою.
- Перевірити можна за допомогою макроса
- `__STDC_HOSTED__` – дорівнює 1 для hosted та 0 інакше.
- Freestanding-реалізація:
  - ◇ Не обов'язково підтримує потоки.
  - ◇ Не обов'язково потребує main(), код ініціалізації і завершення виконання – на розсуд реалізації.
  - ◇ Частина стандартних файлів заголовків обов'язково присутня, частина – повинні бути лише частково реалізовані, решта – можуть бути відсутніми.
  - ◇ Приклад: <cstddef> має бути весь, <cstdlib> – частково, <vector> може бути відсутній – через динамічну пам'ять та виключення.
  - ◇ В нових стандартах кількість хедерів, обов'язкових в freestanding реалізаціях, зростає.
- Детальніше див. `https://en.cppreference.com/w/cpp/freestanding`.

- Експериментуйте.
- `https://embeddedartistry.com/blog/2019/04/17/exploring-startup-implementations-newlib-arm/`

